

سند استاندارد ایزوله‌سازی مستأجران

Tenant Isolation & Multi-Tenant Security Standard

نسخه: ۱٫۰ (Freeze)

وضعیت - Architectural Decision Document (ADD): مصوب و نهایی

مرجع فنی: اکوسیستم ۱۱.x Laravel و ۱۶+ PostgreSQL

=====

=====

۱. هدف سند

این سند قوانین، اصول و الزامات زیرساختی مربوط به حفظ جداسازی اطلاعات مشتریان (Tenant Isolation) را در معماری چندمستأجری (Multi-Tenant) سیستم HamarehERP تعریف می‌کند. هدف اصلی این استاندارد، جلوگیری از هرگونه نشت اطلاعات (Data Bleed) بین مشتریان و تضمین امنیت داده‌ها در تمام لایه‌های دیتابیس، بک‌اند، کلاینت و سیستم‌های جانبی است. با توجه به اینکه معماری انتخابی سیستم مدل "Shared Database Multi-Tenant" (پایگاه‌داده مشترک با تفکیک منطقی) است، رعایت دقیق ابعاد این سند یک اصل حیاتی، غیرقابل مذاکره و متضمن پایداری کل پلتفرم است.

=====

=====

۲. اصل پایه Tenant Isolation

هر Tenant باید به عنوان یک فضای داده‌ای و امنیتی کاملاً مستقل، مجزا و خودمختار در نظر گرفته شود.

هیچ کاربر، سرویس، API، پروسس پس‌زمینه یا ماژولی تحت هیچ شرایطی نباید بتواند به داده‌های متعلق به یک Tenant دیگر دسترسی پیدا کند، مگر با مجوزهای صریح و تعریف‌شده در لایه SaaS Admin پلتفرم.

نشت اطلاعات بین مستأجران (Data Bleed / Data Bleeding) یک خطای بحرانی درجه یک معماری (Critical Architectural Flaw) محسوب شده و منجر به توقف فوری خط توسعه خواهد شد.

۳. قانون Tenant Context

تمام درخواست‌های ورودی به سیستم (Requests)، جاب‌های پس‌زمینه (Jobs) و رویدادها (Events) باید دارای یک بافتار مستأجر (Tenant Context) معتبر، احراز هویت شده و فریز شده باشند.

Tenant Context در بدو ورود مشخص می‌کند:

- درخواست دقیقاً متعلق به کدام مستأجر (Tenant UUID) است.
 - کاربر فعلی در کدام فضای سازمانی (Company / Branch / Department) مجاز به فعالیت است.
 - محدوده عملیاتی (Scope Driven Access) مجاز روی داده‌ها چیست.
- هیچ عملیات کسب‌وکاری (Business Operation) بدون وجود و تایید Tenant Context معتبر در سیستم لایف‌سایکل، اجازه اجرا، پردازش یا ذخیره‌سازی ندارد.

۴. قوانین لایه Database (PostgreSQL)

در تمام جداول عملیاتی و پایه‌ای که داده‌های آن‌ها متعلق به مشتری/مستأجر است، رعایت اصول زیر در لایه بانک اطلاعاتی الزامی است:

۴.۱. اجباری بودن فیلد شناسه: وجود فیلد tenant_id از نوع داده‌ای UUID در تمامی این جداول بدون استثنا الزامی است. این فیلد باید به عنوان بخشی از ساختار ایندکس‌های ترکیبی جهت بهینه‌سازی کوئری‌ها در نظر گرفته شود.

۴.۲. الزام ساختاری: PostgreSQL Row-Level Security (RLS)

تیم توسعه مکلف است امنیت جداسازی داده‌ها را مستقیماً در سطح هسته پایگاه داده پورتگرس پیاده‌سازی کند. تکیه بر شرط‌های دستی در سطح کد کافی نیست.

- برای تمامی جداول مستأجرمحور، باید ویژگی RLS فعال شود (ALTER TABLE table_name ENABLE ROW LEVEL SECURITY;).

- یک Policy سراسری در پورتگرس تعریف می‌شود که به صورت خودکار فیلد tenant_id رکورد را با متغیر سشن محلی دیتابیس مقایسه می‌کند.

- در بدو برقرار کانکشن لاراول به دیتابیس پورتگرس، شناسه مستأجر جاری باید در سشن دیتابیس تزریق شود:

```
SET local app.current_tenant_id = 'tenant-uuid';
```

- هرگونه کوئری که این متغیر را دور بزند یا فاقد آن باشد، توسط دیتابیس ریجکت خواهد شد .

=====

=====

۵. قوانین لایه Backend (Laravel)

بک‌اند لاراول مسئول اصلی مدیریت، استخراج و پاس دادن امن Tenant Context به لایه‌های پایینی است .

۵.۱. استخراج امن شناسه مستأجر: تشخیص Tenant تحت هیچ شرایطی نباید بر اساس ورودی‌های مستقیم، قابل دستکاری و خام کلاینت) مانند فیلد متنی در Body رکوئست (انجام شود. شناسه Tenant باید در لایه Middleware و از طریق کانال‌های امن زیر استخراج و قفل شود :

- تحلیل ساب‌دومین معتبر (مانند company.hamareh.com) از روی آدرس رکوئست .
- یا رمزگشایی مدعای (Claim) مخفی و امضا شده درون توکن امنیتی JWT یا هدر اختصاصی و تایید شده توسط API Gateway.

۵.۲. الزام استفاده از: Laravel Global Query Scopes

تمامی مدل‌های Eloquent لاراول که داده‌های مستأجر را حمل می‌کنند، باید از یک Trait سراسری به نام MultiTenantScope ارث‌بری کنند. این Trait در متد booted مدل، یک فیلتر سراسری را قفل می‌کند تا به صورت خودکار شرط WHERE tenant_id = current_tenant_id را به تمام کوئری‌های صادر شده توسط Eloquent یا Query Builder اضافه کند. این امر ریسک فراموشی برنامه‌نویسان را به صفر می‌رساند .

۵.۳. ایزوله‌سازی متدهای لایه سرویس و مخازن: تمام کلاس‌های Service Layer و Repository ها

باید Tenant Context را به صورت تزریق وابستگی (Dependency Injection) یا از طریق کانتینر مرکزی لاراول (app('tenant')) دریافت کنند. اجرای کوئری‌های خام یا آزاد بدون فیلتر مستأجر ممنوع است .

۶. قوانین لایه API

- تمامی API های مرتبط با داده های پلتفرم باید تحت ضوابط زیر طراحی شوند :
- هر اندپوینت باید در لایه روتینگ خود مجهز به Middleware احراز دسترسی مستأجر (tenant.auth) باشد .
 - API باید پس از تایید صحت مستأجر، معتبر بودن دسترسی کاربر (User Permission) ، نقش کاربر (Role) و محدوده سازمانی او (Scope) را بسنجد .
 - لایه فرانت اند (Frontend) به هیچ عنوان منبع امنیت سیستم محسوب نمی شود؛ تمام اعتبارسنجی ها باید مجدداً در لایه API و بک اند تایید شوند .

۷. قوانین لایه Cache (Redis)

- سیستم حافظه موقت (Cache) پلتفرم باید کاملاً Tenant Aware طراحی شود .
- هیچ داده ای نباید بین مستأجران به دلیل اشتراک فضا در Cache قابل مشاهده یا تداخل باشد .
 - تمامی کلیدهای ذخیره سازی در Redis باید به صورت اجباری مجهز به پیشوند شناسه مستأجر باشند. الگوی استاندارد نام گذاری کلیدها به این شرح است:
tenant:{tenant_id}:{module_name}:cache_key
 - پکیج مدیریت کش لاراول باید به گونه ای پیکربندی شود که این پیشوند را به صورت اتوماتیک در زمان تفکیک کانفکس اعمال کند .

۸. قوانین لایه Event و Queue

- بر اساس ساختار ارتباطی ناهمگام سیستم :
- تمامی رویدادهای پلتفرم (Business & Domain Events) باید به صورت اجباری فیلدهای tenant_id و ساختار دیتای tenant_context را در Payload خود حمل کنند .

- لیسنرهای صف (Queue Consumers) مکلفند در ثانیه اول پردازش هر جاب (Job) یا ایونت توسط دستور `php artisan queue:work`، ابتدا لایف سائیکل و `Tenant Context` مربوط به آن رویداد را در بک اند بازسازی و قفل کنند و سپس منطق بیزینس را جلو ببرند. پردازش هرگونه ایونت بیزینسی بدون مشخص بودن بافتار مستأجر ممنوع است .

=====

=====

۹. قوانین Audit Trail و Logging

تمام لاگ های عملیاتی، خطاهای سیستم (Exceptions) و رکوردهای حسابرسی سیستم باید قابلیت ردیابی مستأجر را داشته باشند .

- ساختار سیستم لاگینگ لاراوول (به وسیله کانال مانیتورینگ اختصاصی) باید به گونه ای تنظیم شود که متغیر `tenant_id` را به عنوان یک Context جفت شده به تمامی لاگ ها اضافه کند .
- هدف از این ساختار: بررسی رخدادهای دقیق سازمانی، عیب یابی مایکرو (Debugging) ، امنیت پلتفرم و ارائه گزارش های حسابرسی (Audit Trail) به مدیران ارشد هر مستأجر است .

=====

=====

۱۰. تست های امنیتی الزامی (Automated Security Testing)

در خط لوله تست پلتفرم (CI/CD) ، سناریوهای تست نفوذ خودکار زیر باید در هر کامیت بررسی شوند و در صورت بروز خطا، بیلد پروژه باید ریجکت شود :

- تلاش برای دسترسی کاربر Tenant A به شناسه یا داده های عملیاتی متعلق به Tenant B (تست Data Bleed).
- ارسال مستقیم درخواست های API با استفاده از مخدوش کردن هدر یا Tenant ID جعلی .
- دستکاری ساختار توکن امنیتی JWT جهت تغییر Context مستأجر .
- تلاش برای خواندن کلیدهای حافظه موقت (Cache) مستأجران دیگر از طریق دور زدن پیشوندها .
- پردازش موازی یا اشتباه یک Event مربوط به مستأجری دیگر در صف های عمومی پلتفرم .

=====

=====

۱۱. قانون نهایی

مکانیسم Tenant Isolation یک قابلیت یا ویژگی فرعی (Feature) نیست که در آینده به سیستم اضافه شود یا بر حسب سلیقه فعال و غیرفعال گردد. این مفهوم، ستون فقرات و اصل پایه معماری سیستم HamarehERP است و تمام طراحی‌های فعلی و مازول‌های آینده بدون هیچ استثنایی مکلف به تبعیت ۱۰۰٪ از اصول مصوب این سند هستند .